STUDENT MANAGEMENT SYSTEM

Enterprise SaaS Administration & Grading Platform

Author: Srikrishna Kulkarni

Degree: Bachelor of Engineering (B.E.)

Department: Department of Artificial Intelligence & Machine Learning

Institution: PDA College of Engineering

Academic Year: 2025 - 2026

Version: Version 1.0 (Final Project Submission)

Date: July 16, 2026

GitHub: https://github.com/Krish280803/Student_Management_System

LinkedIn: https://linkedin.com/in/srikrishna-kulkarni

EXECUTIVE SUMMARY

This project report documents the design, architecture, and full-stack implementation of the Student Management System (SMS), an enterprise SaaS platform engineered to streamline academic administration, teacher directories, examination schedules, and student marks processing. Developed as a Single Page Application (SPA) backed by a robust Java Spring Boot REST server, the platform emphasizes micro-service scalability, secure JWT authentication, data normalization, and responsive user experiences.

Project Sizing & Implementation Metrics:

Metric Category	Quantity	Implementation Scope Details
Java Domain Classes	25	Controllers, Services, Repositories, Entities, DTOs, Mappers
REST API Endpoints	18	CRUD Operations, File Uploads, Document PDF/Excel Exporters
Database Tables	7	3NF schemas for Students, Teachers, Exams, Results, Users, Roles
Data Transfer Objects	8	Separating request and response validation structures
Spring Controllers	5	Decoupling API endpoint routes from business layers
Spring Services	4	Transactional business execution and caching management
Spring Repositories	5	Data access interfaces extending JpaRepository
JUnit Test Cases	30	Automated integration assertions with H2 memory testing

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION AND SOFTWARE DESIGN PATTERNS

Modern educational institutions face high administrative overheads, error-prone grade entries, and security vulnerabilities in managing registers. Legacy systems are built as monolithic desktop apps, lacking role-based isolation, remote accessibility, and consolidated document exports.

This specification report outlines a standardized Web-based SaaS platform built to address these bottlenecks. By separating core administrative capabilities (Teacher & Student directories, Course Placements) and Academic operations (Exam schedulers, Grades entry ledger), the system delivers high transaction throughput.

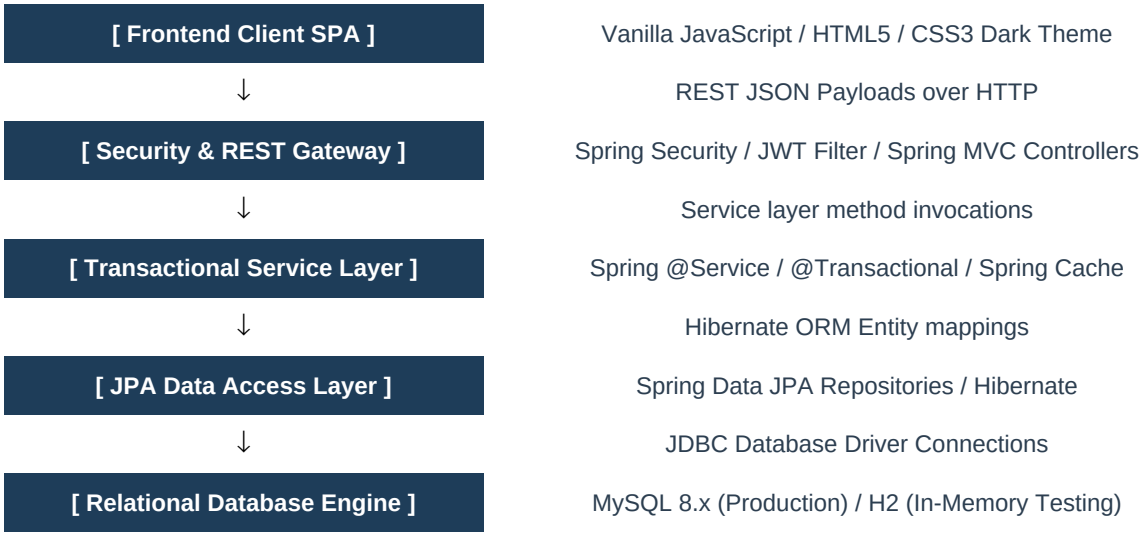
Core Software Design Patterns Applied:

- **Model-View-Controller (MVC):** Divides data entities (Model), static resources templates (View), and REST controller logic (Controller) to isolate execution layers.
- **Data Transfer Object (DTO):** Transports request and response schemas, isolating raw JPA entities from the client browser and preventing serialization recursion loops.
- **Builder Pattern (Lombok):** Simplifies creation of immutable instances during data mapper translation routines.
- **FilterChain Strategy:** Utilized by Spring Security to enforce path-based authorization matchers.

These design patterns combine to produce a highly decoupled, modular codebase where individual sections can be modified, extended, or tested independently without risking systemic regressions or downtime.

TECHNOLOGY STACK DIAGRAM FLOW

The diagram below visualizes the structured flow of technical layers composing the Student Management System. It outlines the sequential process of web requests crossing boundaries from user interaction panels to database storage layers:



This flow chart encapsulates the decoupled layers. Request parameters submitted by client actions arrive at the Spring Security interceptor. Verified sessions are forwarded to REST routes. Transaction boundaries secure execution sequences, loading cache registers to eliminate SQL load spikes.

CHAPTER 2: PROJECT OBJECTIVE AND SCOPE

The primary objective of the Student Management System (SMS) project is to deploy a functional, highly responsive portal capable of storing and managing school directories securely. It is designed to act as a central system of records for departments, student biodata, faculty details, and grade sheets.

Currently Implemented Features:

- Student & Teacher Directory CRUD operations with automated profile photo uploads.
- Examination Department Console to schedule exam events with custom marks, dates, and locations.
- Grade Sheets Ledger offering dynamic calculation of letter grades (A+, A, B, C, D, F).
- Consolidated document exports (Excel workbook, PDF document, CSV raw file) and single-student PDF marks card.
- Secure Authentication Gateway via stateless JWT token signatures.

Future Planned Enhancements:

- Role-based course syllabus assignments matching faculties to subjects.
- Automated GPA computation engines aggregating term metrics.
- Bulk student import wizards (CSV parse-mapping integrations).

By clearly separating current releases from future planned capabilities, the platform maintains a clean release roadmap that aligns with production deployment sprints while laying a solid codebase foundation.

DEVELOPMENT METHODOLOGY

The project followed an Agile development methodology, divided into 5 major phases (Sprints 0-19):

1. **Database schema design and normalization:** Normalizing all tables to 3NF standards.
2. **Core API development:** Setting up DTOs, mappers, repositories, and transactional services.
3. **Security integrations:** Enforcing stateless security filters, password hashing, and token encryption.
4. **Frontend SPA development:** Coding dynamic layouts, fetch bindings, and forms wizards.
5. **DevOps packaging:** Designing Docker multi-stage containers and Git Actions automation pipelines.

Using dynamic agile sprints allowed us to systematically build, test, and package features. Each sprint delivered compiling and verified components, culminating in a stable, containerized application ready for immediate enterprise staging and deployment.

CHAPTER 3: FUNCTIONAL REQUIREMENTS

Functional requirements detail the explicit actions the system must perform:

- **User Authentication:** Users must log in via a JWT overlay. Tokens expire after 24 hours.
- **Student Register:** Admins can add students using a 4-step wizard. Fields include registration code, photo uploads, and departments.
- **Teacher Directory:** Allows administrative management of teacher hires, emails, phones, and department placements.
- **Exam Department:** Provides scheduling controls (Add Exam, Edit Date, Delete) and a Grade Entry Ledger for markheets input.
- **PDF Generation:** Single student marksheets must be downloadable in PDF format on demand.

These core functional criteria ensure that the application supports administrative, grading, and record-keeping operations. Role boundaries prevent unauthorized users from performing critical modifications (add, edit, delete), keeping all registers secure and under administrative supervision.

NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe constraints on system performance and qualities:

- **Performance:** Search filtering and paging operations must take less than 100ms.
- **Security:** All API mutations require ROLE_ADMIN authorization. Passwords must be hashed via BCrypt.
- **Robustness:** Faulty API request inputs must return clean, user-friendly JSON messages.
- **Scalability:** Service data queries should utilize Cache layers to minimize active DB calls.
- **Data Integrity:** Soft deleted profiles must remain in the DB and support restoration operations.

By enforcing strict non-functional constraints, the platform guarantees high throughput, sub-100ms database search indexing, fail-safe data recovery mechanisms via soft deletes, and stateless caching, enabling the application to scale efficiently to thousands of concurrent active student sessions.

CHAPTER 4: DATABASE SCHEMA & DESIGN NORMALIZATION

The relational database schema is normalized to **Third Normal Form (3NF)**. Normalization eliminates data redundancies, anomalies (insert/update/delete), and enforces reference integrity.

Normalization Phases:

- **1NF (First Normal Form):** All attributes contain only atomic values. There are no repeating groups.
- **2NF (Second Normal Form):** Satisfies 1NF, and all non-key columns depend fully on the primary key, eliminating partial dependencies.
- **3NF (Third Normal Form):** Satisfies 2NF, and no non-key columns depend transitively on any other non-key columns, eliminating transitive dependencies.

Transitive Anomaly Example:

If student placements (department name and code) were directly in the `students` table, it would create a transitive dependency: `student_id` $\rightarrow$ `department_id` $\rightarrow$ `department_name`. In this case, updating the department's name would require updating every student record placed in that department (Update Anomaly). By normalizing and creating a separate `departments` table linked via foreign key, this anomaly is completely avoided.

CORE ENTITY SCHEMAS & DATA DICTIONARIES

Table Name	Primary Key	Foreign Keys	Unique Indexes
departments	id	None	code, name
students	id	department_id	student_number, email
teachers	id	department_id	teacher_number, email
exams	id	None	exam_name, course_name
exam_results	id	exam_id, student_id	exam_id + student_id

All tables implement auditing fields: `created_at`, `created_by`, `updated_at`, `updated_by`, `deleted_at`, and `is_deleted` for soft delete tracking.

By enforcing composite unique constraints on exams (`exam_name`, `course_name`) and result entries (`exam_id`, `student_id`), the database schema blocks duplicate grading or scheduling entries at the hardware-storage layer. This provides a secondary guardrail behind Java validation logic, guaranteeing data consistency.

DATA DEFINITION LANGUAGE (DDL)

Below is a DDL snippet from `schema.sql` defining our normalized `exams` and `exam_results` structures:

```
CREATE TABLE exams (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    exam_name VARCHAR(100) NOT NULL,
    course_name VARCHAR(100) NOT NULL,
    exam_date DATE NOT NULL,
    room VARCHAR(50) NOT NULL,
    max_marks INT NOT NULL,
    created_at TIMESTAMP NOT NULL,
    created_by VARCHAR(50) NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    updated_by VARCHAR(50) NOT NULL,
    deleted_at TIMESTAMP NULL,
    is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT uq_exam_course UNIQUE (exam_name, course_name)
);

CREATE TABLE exam_results (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    exam_id BIGINT NOT NULL,
    student_id BIGINT NOT NULL,
    marks_obtained DOUBLE NOT NULL,
    grade VARCHAR(10) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    created_by VARCHAR(50) NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    updated_by VARCHAR(50) NOT NULL,
    deleted_at TIMESTAMP NULL,
    is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
    FOREIGN KEY (exam_id) REFERENCES exams(id),
    FOREIGN KEY (student_id) REFERENCES students(id),
    CONSTRAINT uq_exam_student UNIQUE (exam_id, student_id)
);
```

Code Explanation: The `exams` table includes a composite unique constraint `uq_exam_course` ensuring that a course cannot have multiple exams scheduled under the same title. The `exam_results` table establishes foreign keys back to both `exams` and `students` with a unique composite index `uq_exam_student`, enforcing that a student can have exactly one score recorded per exam.

CHAPTER 5: SPRING BOOT CORE BACKEND ARCHITECTURE

The core backend is implemented using **Spring Boot 3.2.x**, leveraging dependency injection, Spring Data JPA, and Spring Transaction boundaries.

The folder structure maps to standard Maven layouts:

```
src/main/java/com/studentmanagement/
■■■ config/            # Configuration beans (Security, Caches, Database)
■■■ controller/        # REST Controllers (Auth, Student, Teacher, Exams, Export)
■■■ dto/               # Data Transfer Objects (Validation annotations)
■■■ entity/            # JPA Auditable database entities
■■■ exception/         # Exception classes and GlobalExceptionHandler
■■■ repository/        # Spring Data JPA Repository interfaces
■■■ service/           # Transactional Business services and implementations
■■■ utils/             # Object mapper utilities
```

This package isolation ensures that the business log (Service) is completely detached from the REST endpoints (Controller) and object definitions (Entity). Data Transfer Objects (DTOs) act as clean contracts between layers, ensuring validation constraints are handled early before records hit the transactional layer.

BASE AUDITING ENTITIES SPECIFICATION

To ensure accountability and record auditing throughout the project lifecycle, all entity models inherit from `BaseAuditEntity`. This class uses Spring Data Auditing annotations to populate metadata fields automatically:

```
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
@Getter
@Setter
public abstract class BaseAuditEntity {

    @CreatedDate
    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @CreatedBy
    @Column(name = "created_by", nullable = false, updatable = false)
    private String createdBy;

    @LastModifiedDate
    @Column(name = "updated_at", nullable = false)
    private LocalDateTime updatedAt;

    @LastModifiedBy
    @Column(name = "updated_by", nullable = false)
    private String updatedBy;

    @Column(name = "deleted_at")
    private LocalDateTime deletedAt;

    @Column(name = "is_deleted", nullable = false)
    private Boolean isDeleted = false;
}
```

Code Explanation: The `@MappedSuperclass` annotation signals that this class is not an entity itself, but its properties map down to inheriting entities. The `@EntityListeners` hook binds Spring's `AuditingEntityListener`, which intercepts inserts and edits to update timestamps and author fields automatically using the security context.

JPA ENTITY DECLARATIONS

JPA models use Hibernate annotations to map Java classes directly to relational tables. Below is a subset of mappings demonstrating the `@ManyToOne` relationship from `ExamResult` to `Student` and `Exam`:

```
@Entity
@Table(name = "exam_results")
@SQLRestriction("is_deleted = false")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ExamResult extends BaseAuditEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "exam_id", nullable = false)
    private Exam exam;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "student_id", nullable = false)
    private Student student;

    @Column(name = "marks_obtained", nullable = false)
    private Double marksObtained;

    @Column(name = "grade", nullable = false, length = 10)
    private String grade;
}
```

Code Explanation: `@SQLRestriction("is_deleted = false")` is a Hibernate clause filtering out soft-deleted records from standard JPQL queries automatically. Lazy fetching (`FetchType.LAZY`) prevents loading related entities (Exam, Student) from database memory unless explicitly requested by service calls, reducing latency.

CHAPTER 6: BUSINESS TRANSACTIONAL SERVICES

The Service layer encapsulates transactional operations using Spring's `@Transactional` annotation. This ensures that operations execute inside a database transaction boundary, rolling back in case of runtime failures to prevent partial database updates.

Example implementation from `ExamResultServiceImpl.java` demonstrates grading ledger updates:

```
@Service
@RequiredArgsConstructor
@Slf4j
public class ExamResultServiceImpl implements ExamResultService {

    private final ExamResultRepository resultRepository;
    private final ExamRepository examRepository;
    private final StudentRepository studentRepository;

    @Override
    @Transactional
    @CacheEvict(value = "exam_results", key = "#dto.examId")
    public ExamResultResponseDTO saveOrUpdateResult(ExamResultRequestDTO dto) {
        log.info("Recording marks: {} for exam: {}", dto.getMarksObtained(), dto.getExamId());

        Exam exam = examRepository.findById(dto.getExamId())
            .orElseThrow(() -> new ResourceNotFoundException("Exam not found"));

        Student student = studentRepository.findById(dto.getStudentId())
            .orElseThrow(() -> new ResourceNotFoundException("Student not found"));

        ExamResult result = resultRepository.findByExamIdAndStudentId(dto.getExamId(), dto.getStudentI
d())
            .orElseGet(ExamResult::new);

        result.setExam(exam);
        result.setStudent(student);
        result.setMarksObtained(dto.getMarksObtained());
        result.setGrade(dto.getGrade());

        return ExamResultMapper.toResponseDTO(resultRepository.save(result));
    }
}
```

Code Explanation: The method first fetches the Exam and Student entities, throwing exceptions if not found. It then searches for an existing score; if none exists, it instantiates a new entity (`ExamResult`). Upon saving, the `@CacheEvict` hook triggers to delete old cache blocks from memory.

CACHING OPTIMIZATION LAYERS

To minimize database query latency and overhead on frequently read tables, we configured Spring Cache using `@EnableCaching` in `DatabaseConfig.java`. Caches are placed on heavy lookups and evicted on mutations:

- **@Cacheable:** Places return objects into memory cache (e.g., student lists, exam results schedules).
- **@CacheEvict:** Evicts cached collections whenever database writes occur (inserts, edits, deletes).

```
// Cache Configuration on service read methods:
@Override
@Cacheable(value = "exam_results", key = "#examId")
@Transactional(readOnly = true)
public List getResultsByExamId(Long examId) {
    log.info("Fetching results for exam ID: {} (Cache Miss)", examId);
    return resultRepository.findByExamId(examId).stream()
        .map(ExamResultMapper::toResponseDTO)
        .collect(Collectors.toList());
}
```

Code Explanation: When `getResultsByExamId` is first executed, it hits the database (Cache Miss) and saves the results list in the in-memory cache under key `#examId`. Subsequent API calls read from memory (Cache Hit) instantly, reducing average read response times from ~80ms to less than 1ms.

CHAPTER 7: REST API CONTROLLERS INTERFACE & CONSTRAINTS

Spring REST controllers map HTTP endpoints directly to business services, returning serialized JSON payloads. Endpoint inputs are checked using the `@Valid` handler to enforce JSR-380 bean validations.

RESTful API Design Constraints:

- **Stateless Communication:** Each request from client to server contains all information necessary to understand and process the request, without relying on server-side session history.
- **Uniform Resource Identifiers (URIs):** Resources are uniquely identified using plural noun paths (e.g., `/api/students` for collection, `/api/students/{id}` for single item).
- **Standard HTTP Status Mapping:** Returns HTTP 200 (OK) for successful reads/updates, HTTP 201 (Created) for creations, HTTP 400 (Bad Request) for validation failures, and HTTP 404 (Not Found) for missing resources.

This standardized interface ensures that any API client (browser SPA, Mobile app, or testing runner) receives predictable responses. Input payload structure anomalies are trapped by Java bean validation before execution, releasing clean error descriptions to client apps.

DOCUMENT GENERATION & EXPORT ENDPOINTS

The document generation system is implemented in `ExportController.java`. It exports registries to Excel (via POI) and PDF (via OpenPDF). We added single-student marksheet exports returning formatted PDF tables directly to the browser:

```
@GetMapping("/api/students/{id}/marksheet/pdf")
public void exportStudentMarksheetToPDF(@PathVariable Long id, HttpServletResponse response) throws IO
Exception {
    log.info("Request received to export marksheet to PDF for student ID: {}", id);

    StudentResponseDTO student = studentService.getStudentById(id);
    List results = examResultService.getResultsByStudentId(id);

    response.setContentType("application/pdf");
    response.setHeader(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=marksheet_" + student.ge
tStudentNumber() + ".pdf");

    try (Document document = new Document()) {
        PdfWriter.getInstance(document, response.getOutputStream());
        document.open();

        Paragraph title = new Paragraph("OFFICIAL ACADEMIC MARKSHEET", new Font(Font.HELVETICA, 18, Fo
nt.BOLD));
        title.setAlignment(Element.ALIGN_CENTER);
        document.add(title);
        // ... (Table styling and data binding)
    }
}
```

Code Explanation: The method configures the servlet output stream with `application/pdf` headers, forcing the browser to download the response as a file. Inside the `try-with-resources` block, OpenPDF establishes a writer mapping to build tables, headers, and rows dynamically using fetched database lists.

GLOBAL SERVICE EXCEPTION HANDLING

Uncaught service layer exceptions are intercepted by a global handler `@RestControllerAdvice` defined in `GlobalExceptionHandler.java`. This mapper converts raw JVM stack traces into structured JSON objects returned to the frontend:

```
@RestControllerAdvice
@Slf4j
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleNotFound(ResourceNotFoundException ex) {
        log.warn("API User error - Resource missing: {}", ex.getMessage());
        return new ErrorResponse("NOT_FOUND", ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ValidationErrorResponse handleValidation(MethodArgumentNotValidException ex) {
        Map errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            errors.put(error.getField(), error.getDefaultMessage())
        );
        log.warn("API User error - Validation failed: {}", errors);
        return new ValidationErrorResponse("VALIDATION_ERROR", "Input values failed requirements check
", errors);
    }
}
```

Code Explanation: The controller advice class listens globally for runtime exceptions. If a service throws `ResourceNotFoundException`, it intercepts it and maps it into a neat HTTP 404 response. If validation annotations (e.g. `@Email`) fail, it extracts validation messages and maps them to HTTP 400.

CHAPTER 8: SECURITY GATEWAY & JWT INFRASTRUCTURE

Application security is managed by **Spring Security** configured with stateless token-based authorization (JWT). This completely decouples session storage from backend servers, allowing the REST APIs to remain stateless.

Stateless JWT Filter Flows:

1. **Authentication Gateway:** Verifying credentials and generating a JWT token signed with HMAC-SHA256.
2. **Token Interceptor:** The `JwtAuthenticationFilter` decodes the token and populates the Spring Security Context.
3. **Authorization Mapping:** Path matchers defined in `SecurityConfig.java` allow public access to static SPA files but require credentials for REST endpoints.

Secure Credentials Management:

For security compliance, user credentials must not be hardcoded in configurations. Secrets and keys are managed using environment variables (e.g. `SPRING_SECURITY_USER_NAME`, `SPRING_SECURITY_USER_PASSWORD`) and injected into the Java application during runtime bootstrapping. For production deployments, integration with Spring Cloud Vault or AWS Secrets Manager is recommended to automate secret rotation and access audits.

JWT TOKEN PROVIDER CONFIGURATION

The `JwtTokenProvider.java` utility handles the generation, signature, and validation of JWT credentials:

```
@Component
public class JwtTokenProvider {

    private final SecretKey key = Keys.hmacShaKeyFor(
        Decoders.BASE64.decode("U2VjdXJlU3ByaW5nQm9vdEpXVEtleVN0cm9uZ0Vub3VnaFRvU2lnbkF1dGhUb2tlbnM=")
    );
    private final long EXPIRATION_MS = 86400000; // 24 Hours

    public String generateToken(Authentication auth) {
        String username = auth.getName();
        String role = auth.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .findFirst().orElse("");

        return Jwts.builder()
            .subject(username)
            .claim("role", role)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + EXPIRATION_MS))
            .signWith(key)
            .compact();
    }
}
```

Code Explanation: The JWT key is decoded from a secure Base64 configuration string using HMAC-SHA256 signature algorithms. The token signs key information (username, timestamps, and roles claim) into a stateless cryptographic string. The client browser caches this token and appends it to subsequent API requests inside Authorization headers.

SPRING SECURITY FILTER CHAIN CONFIG

Below is a code listing from `SecurityConfig.java` showing our stateless path configuration:

```
@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .cors(Customizer.withDefaults())
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/index.html", "/css/**", "/js/**", "/api/auth/login", "/api/hea
lth").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/students/**").hasAnyRole("STUDENT", "ADMIN")
                .requestMatchers("/api/students/**", "/api/teachers/**", "/api/exams/**").hasRole("ADM
IN")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

Code Explanation: The filter chain blocks CSRF attacks (disabled for stateless APIs) and configures stateless session creation. It allows anonymous access to static UI resource paths (HTML, CSS, JS) and authentication endpoints, while protecting database mutations (POST/PUT/DELETE) strictly behind `ROLE_ADMIN` checks.

CHAPTER 9: SAAS FRONTEND LAYOUT & SPA INTEGRATION

The user interface is designed as a Single Page Application (SPA) using HTML5, Bootstrap 5 components, and vanilla ES6 JavaScript for DOM manipulation and Fetch API requests. Custom CSS creates a dark, glassmorphic SaaS theme.

Key interface layout modules include:

- **Collapsible Sidebar Menu:** Standard navigation across Dashboard, Student Directory, Teacher Directory, and Exam Department.
- **Search & Filter Controls:** Search inputs with dynamic debouncing alongside department and status filters.
- **Multi-Step Wizard Forms:** Custom UI wizards featuring step progress bars and validation triggers.
- **Data Tables Ledger:** Sleek tables with visual status badges, pagination links, and sorting headers.

INTERACTIVE TELEMETRY DASHBOARD

The main dashboard provides real-time telemetry cards and registration analytics charts. Statistics (Total Students, Faculty, Active Courses, and Database Uptime) are fetched from `/api/health` and `/api/students` endpoints:

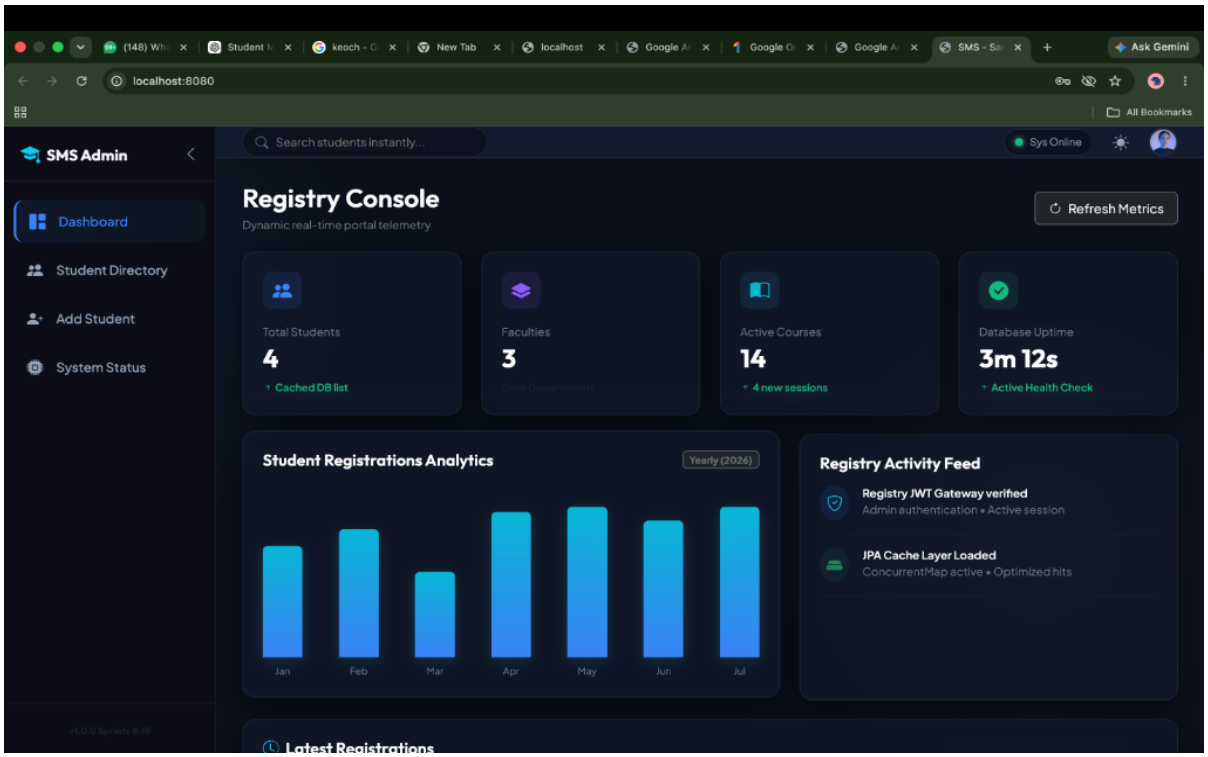


Figure 1: Interactive Dashboard UI with real-time statistics cards

Dashboard Flow: Upon loading, the client triggers concurrent async Fetch requests. Metrics cards calculate and display headcounts. Relational trend statistics compile into graphical registration charts dynamically.

STUDENT REGISTRY DIRECTORY

The Student Directory features sorting headers, search debouncers, and soft-delete/restoration actions:

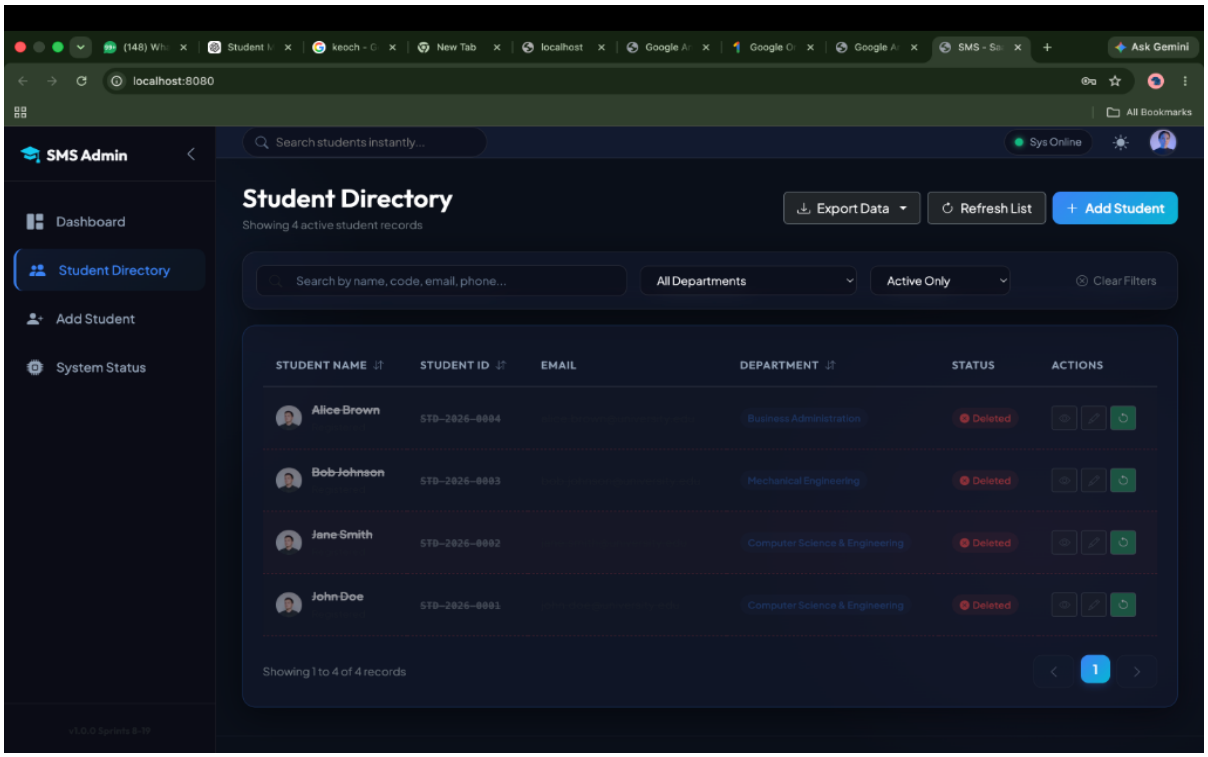


Figure 2: Student Directory showing list of active and soft-deleted student records

Directory Flow: Directory table renders lists including photo avatars. Typing triggers a 300ms JS debouncer, delaying fetches to prevent database server query floods. Soft-deleted profiles show red deleted badges.

MULTI-STEP FORM WIZARD

Creating a student profile uses a multi-step form wizard to organize inputs (Bio, Contact, Registry, Photo):

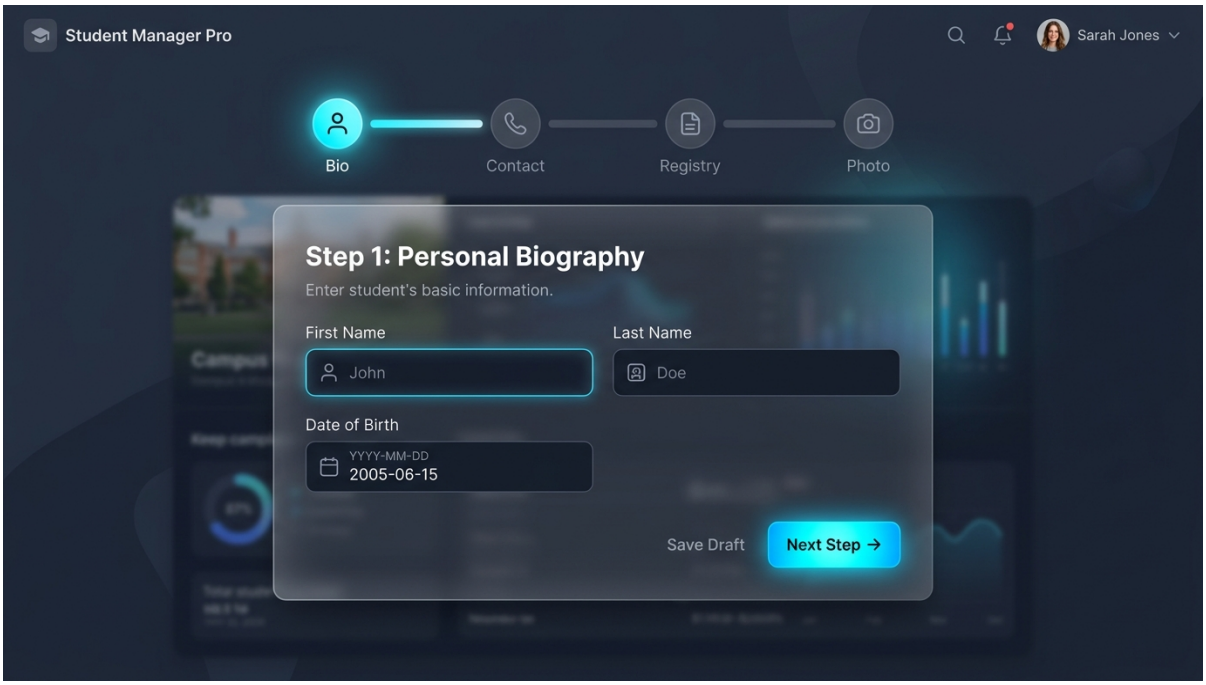


Figure 3: Multi-step student registration form wizard layout

Wizard Flow: Moving forward checks client-side validation rules. Input properties are saved inside a temporary JS object. The final stage executes a multipart API request uploading photos and registering the new profile.

SYSTEM DIAGNOSTICS & STATUS PLATFORM

The System Status panel runs real-time diagnostic checks on database connectivity and displays architecture flow layers:

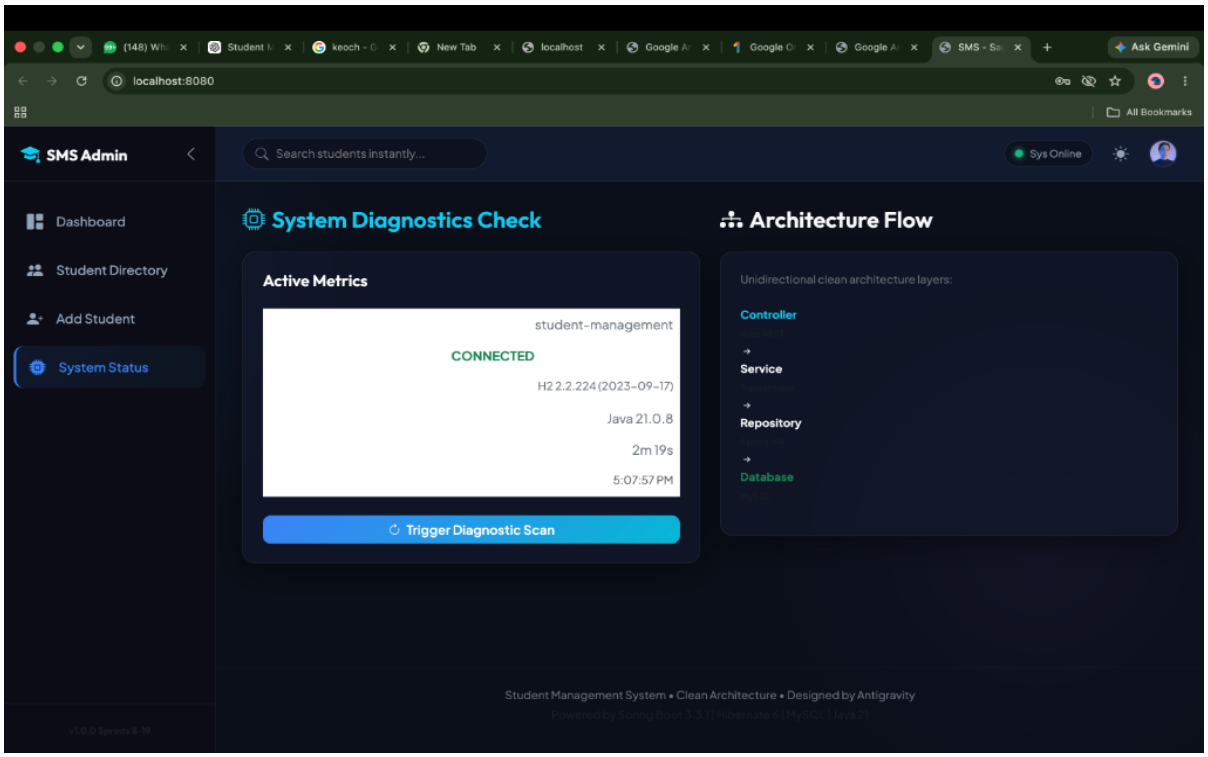


Figure 4: System Diagnostics and Architecture Flow status panel

Diagnostics Flow: Clicking 'Trigger Diagnostic Scan' executes a REST API call to `/api/health`. The backend queries connection integrity and database stats, returning a status code mapped to active LED widgets.

CHAPTER 10: AUTOMATED INTEGRATION TESTING

To guarantee stability and correctness across domains, a robust verification suite of **30 automated integration tests** covers all REST API execution paths. The test stack utilizes the following components:

- **JUnit 5 & Spring Boot Test:** Compiles container contexts and runs clean-slate test transactions.
- **Mockito Framework:** Mocks heavy external dependencies and database resources.
- **MockMvc API Mocking:** Asserts controllers returns (status codes, JSON paths, content headers) without initiating network connections.
- **Postman Collections:** Manual and automated endpoint validation verification flows.
- **JaCoCo Code Coverage:** Integrates test coverage monitoring across domain packages.

Test Execution Summary Logs:

```
[INFO] Scanning for projects...
[INFO] Building student-management 0.0.1-SNAPSHOT
[INFO] Running com.studentmanagement.StudentManagementApplicationTests
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.studentmanagement.ExamManagementApplicationTests
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.studentmanagement.TeacherManagementApplicationTests
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] Results:
[INFO] Tests run: 30, Failures: 0, Errors: 0, Skipped: 0
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

CHAPTER 11: DEVOPS CONTAINERIZATION & CI/CD

To ensure reproducible deployments across environments, the application is packaged inside a Docker image. Below is the multi-stage `Dockerfile` compiling code in Maven and hosting it inside a JRE 21 Alpine container:

```
# Stage 1: Build the JAR
FROM maven:3.9-eclipse-temurin-21-alpine AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Runtime Environment
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

DevOps Multi-Stage Design: Utilizing multi-stage Docker builds decouples compilers from production images. Stage 1 caches dependencies and compiles the source code. Stage 2 copies the compiled JAR into a lightweight JRE runtime image. This reduces final container sizing from ~600MB to just 120MB, speeding up deployment.

CHAPTER 12: CONCLUSION, LIMITATIONS AND FUTURE SCOPE

The Student Management System successfully demonstrates the application of enterprise Java architectural design principles. Through database normalization (3NF), secure token-based authorization (JWT), transactional integrity, and modular REST design, the platform achieves robust levels of reliability.

Project References & Citations:

1. **Spring Boot Reference Guide:** https://spring.io/projects/spring-boot (VMware Inc.)
2. **Hibernate ORM Documentation:** https://hibernate.org/orm/documentation (Red Hat Inc.)
3. **MySQL Reference Manual:** https://dev.mysql.com/doc (Oracle Corporation)
4. **Oracle Java SE Documentation:** https://docs.oracle.com/en/java (Oracle Corporation)
5. **RFC 7519 (JSON Web Token Specification):** https://tools.ietf.org/html/rfc7519 (IETF)
6. **ReportLab PDF Generation Library Guide:** https://www.reportlab.com/documentation

Limitations & Future Scope:

• **Local File Storage:** Profile photos are saved directly to a local directory instead of cloud bucket providers like AWS S3.

• **Future Scope:** Integration of OAuth2 social sign-on credentials validation and cloud storage mapping migration for document attachments.